Workshop notebooks: detailed walkthrough

RNA Canada Computational Biology Workshop · TA and instructor guide to the three follow-along notebooks

How to use this guide

This document explains, cell by cell, the three Colab notebooks participants receive: what each step does, what goes in and what comes out, what participants type in each TODO, what a correct result looks like, and what to say in the debrief. One section per notebook. It is meant for TAs and instructors; send participants the notebooks only.

Notebook	File sent to participants	Answer key (keep private)	Time	Runtime
Session 1: Data mining, APIs and web scraping	`Session1_Data_Mining_APIs_Scraping.ipynb`	`Session1_ANSWER_KEY.ipynb`	~65 min core, +20 optional	CPU
Session 2: Machine learning on RNA data	`Session2_Machine_Learning_RNA.ipynb`	`Session2_ANSWER_KEY.ipynb`	~60 min core, +10 optional	CPU
Model confidence: ESMFold and AlphaFold2	`Model_Confidence_Workshop_ESMFold_AlphaFold.ipynb`	`Model_Confidence_ANSWER_KEY.ipynb`	~70 min	T4 GPU (falls back without)

Conventions shared by all three notebooks

Marker in the notebook	What it means for participants
✏ TODO with a `___` blank	Replace the three underscores with one small answer (usually a word, a variable name or a number), then run the cell. Every TODO has exactly the blanks it needs and nothing else to write.
Hint	A comment above the blank that says what to type. Hints are deliberately close to the answer; the audience has little or no coding background.
✔ Stuck? Click here to reveal the answer	A collapsed text cell directly below each TODO with the filled-in line(s). Using it is encouraged: a skipped TODO would break every later cell.
★ Optional	Extra sections for fast participants or after the workshop. Every notebook was run with all optional cells skipped to confirm nothing later depends on them.
Expected result	Many description cells state what a correct output looks like, so participants can check themselves without a TA.
Discussion / Think / Predict first	Prompts for pairs or the room. The prediction cells ask participants to commit to a guess before seeing a result.

Every notebook opens with a timing table and a short "How to use this notebook" cell (save a copy to Drive, Shift+Enter, run top to bottom, restart the session if something breaks). All outputs are cleared in the participant versions, so nobody sees results before predicting them.

Where the reference numbers come from

The "Reference numbers" in this guide come from the earlier Colab runs of the original notebooks, plus an offline test run of the new versions for the Session 2 fallback dataset. The cleaned-up notebooks use the same code for every computation, but live databases change: treat the numbers as what to expect, not exact targets, and write the numbers from your own pre-workshop run into the checklist at the end.

Before the workshop: must-do items

1. **Decide what to do about the RNAcentral search** (affects Session 1 Parts 2.3 and 6, and Session 2 Part 2.1). In the earlier runs the `query` parameter of `rnacentral.org/api/v1/rna` was ignored: "let-7", "miRNA precursor" and "rna_type:lncRNA" all returned the same recent tRNA, rRNA and misc_RNA records. Either fix the request (EBI Search, `ebi.ac.uk/ebisearch/ws/rest/rnacentral`, is the likely route) or present it openly as a lesson in checking results. Session 2's text currently describes the non-coding class as lncRNA, which it is not with the current fetch.
2. **Run all three notebooks once** in a fresh Colab runtime using the answer keys, and note your own reference numbers.
3. **Test the RCSB POST cell** (Session 1 Part 3). It was a placeholder in the original notebook and could not be run during preparation.
4. **Ask participants to run each Setup cell the day before** (installs, and the few-GB ESMFold download), and to have a Google account for Colab.
5. **Check GPU access** for the model-confidence notebook. Without a GPU it switches to saved results automatically, but participants then cannot fold their own sequences.

Session 1: Data mining, APIs and web scraping

Detailed walkthrough · `Session1_Data_Mining_APIs_Scraping.ipynb`

Big picture

Aim. Show participants the main ways biological data gets into Python (a sequence file, public REST APIs, a website's hidden search API, an HTML table, and optionally a browser robot and a PDF), then package one of them into a reusable function that saves a CSV. About 65 minutes of core material plus 20 minutes optional. No GPU, no prior Python.

What participants do. Twelve TODOs, each a single blank: a file name, a variable name, a dictionary lookup such as `r.get("length")`, a tag name. Each part follows the same rhythm: short explanation, run the demo cells, fill the TODO, compare with the expected result, discuss.

What they should leave with. (1) One pattern that covers most data retrieval: *request → `.json()` → list of dictionaries → `pd.DataFrame` → `.to_csv()`*. (2) Read an API's documentation before writing code. (3) A website's own API is faster and sturdier than scraping its HTML. (4) Real data is messy: always look at what came back.

Part	Minutes	What happens
Setup	3	Install Biopython, requests, BeautifulSoup, pandas, pypdf
1 FASTA	10	Parse a 3-record microRNA FASTA file into records and a DataFrame
2 APIs	20	NCBI esearch + esummary (MIR21), UniProt (APP), RNAcentral, save a CSV
3 RCSB PDB	10	Find the search API in the browser, decode it from the URL, POST it
4 Scraping	10	Wikipedia codon tables with BeautifulSoup, clean, save
5 ★ Selenium	10	Optional: headless Chrome searches Wikipedia
6 Capstone	10	`build_rna_dataset()` function, saves `session1_rna_dataset.csv`
7 ★ Docling	10	Optional: text and tables from the Transformer paper PDF

Setup (about 3 min)

What the cell does. Installs `biopython requests beautifulsoup4 lxml pandas pypdf`, imports requests, pandas and BeautifulSoup, and prints `Setup complete.`

What can go wrong. Slow installs when everyone starts together (ask for it the day before). A `ModuleNotFoundError` after installing is fixed with Runtime → Restart session.

Part 1: Parse your first FASTA file (about 10 min)

Aim. Show what a FASTA record is and how a library turns text into objects with named parts.

Demo cell. Writes `sample_mirna.fasta` with three human microRNA precursors (let-7a-1, mir-21, mir-155) and prints it. Written inline so it works offline.

TODO	Participants type	Expected result
1.1 Read the file with `SeqIO.parse`	`"sample_mirna.fasta"`	Three lines: hsa-let-7a-1 length 80, hsa-mir-21 length 72, hsa-mir-155 length 65
1.2 Fill the `sequence` and `length` columns of each row	`str(rec.seq)` and `len(rec.seq)`	`mirna_df`: 3 rows × 4 columns (id, description, sequence, length)

Debrief. What is the difference between `rec.id` and `rec.description`? Why `str(rec.seq)`? (Without it pandas shows the sequence as a tuple of single letters; the original notebook had exactly this display bug.) The list-of-dictionaries recipe introduced here returns in Parts 2.3 and 6.

What can go wrong. `FileNotFoundError`: the cell that writes the file was not run. A Colab-only path such as `/content/...` breaks elsewhere; the notebook now uses the plain file name.

Part 2: Query live biological APIs (about 20 min)

Aim. Introduce APIs as "web addresses that return data for programs", always used in two steps: `requests.get(url, params=...)` then `.json()`. Point to the docs of NCBI E-utilities, UniProt (OpenAPI/Swagger) and RNAcentral.

2.1 NCBI: search, then summarise

Two calls: **esearch** turns a search term into a list of IDs, **esummary** returns details for one ID. The search term is `MIR21[gene] AND human[orgn]`. Each call uses `raise_for_status()` so a failed request stops with a clear message.

TODO	Participants type	Expected result
Send the esearch request	`search_params`	`Gene IDs found: ['406991']`
Turn the esummary reply into a dictionary	`summary_resp.json()`	Name MIR21, Description microRNA 21, Organism Homo sapiens, Chromosome 17

Two ★ optional "peek" cells print the raw dictionaries (`search_json`, `summary_json`) so participants can find `idlist` and see how much more the record holds.

2.2 UniProt: one protein record

TODO	Participants type	Expected result
Request the protein JSON	`uniprot_url`	Protein name: Amyloid-beta precursor protein · Gene(s): ['APP'] · Sequence len: 770

Try-it prompt: swap `P05067` for `P69905` (hemoglobin subunit alpha) or `P04637` (p53).

2.3 RNAcentral: build a table from a list of records

A demo cell downloads `results` (10 records). The TODO builds one dictionary per record; `rnacentral_id` and `description` are given.

TODO	Participants type	Expected result
Fill the `length` and `rna_type` columns	`r.get("length")` and `r.get("rna_type")`	`rnacentral_df`, 10 rows × 4 columns; saved as `rnacentral_let7_search.csv`

Instructor note. In the earlier run the 10 records were misc_RNA, tRNA and rRNA with descriptions such as "None misc RNA", not let-7. The endpoint appears to ignore `query`. If you do not fix it, use it: ask the room "did we get what we asked for?" before moving on. It is a direct lesson in checking returned data.

Debrief. Why `r.get(...)` instead of `r[...]`? (It returns `None` instead of crashing on a missing key.) Which of the three APIs had the clearest documentation?

What can go wrong. HTTP 429 or 5xx when 20 people query at once: wait a few seconds and re-run. NCBI allows about 3 requests per second without an API key.

Part 3: Find and use the RCSB PDB search API (about 10 min)

Aim. Show that many websites use an API behind the scenes, and that you can find and reuse it.

In the browser. Open the long RCSB search URL (entry 1B23). Inspect → Network → Fetch/XHR to watch the page's requests; in the Console, tick every checkbox with one line of JavaScript.

In Python. The page URL already contains the search, URL-encoded, in its `request=` parameter. `urlparse(...).query` cuts the part after `?`, `parse_qs` decodes it, `json.loads` turns the JSON text into a dictionary, and `requests.post(..., json=...)` sends it to `search.rcsb.org/rcsbsearch/v2/query`.

TODO	Participants type	Expected result
Split the URL into parameters; decode the request	`urlparse(page_url).query` and `query_params["request"][0]`	A pretty-printed JSON query (group → terminal node with `entry_id in ["1b23"]`)
POST the decoded query	`decoded_request`	`Status code: 200` and a JSON answer listing entry 1B23

Instructor note. The POST cell could not be run during preparation (and was unfilled in the original). Test it before the workshop. If RCSB rejects a field, removing `request_info` from `decoded_request` is the first thing to try.

Debrief. Compare with scraping the rendered results page: which is faster, and which breaks first when RCSB redesigns its site?

Part 4: Scrape a table with BeautifulSoup (about 10 min)

Aim. When there is no API, read the page itself. A short HTML picture explains `<table>`, `<tr>`, `<th>`, `<td>`.

TODO	Participants type	Expected result
Find every wikitable on the codon-table page	`"table"`	`Found 8 wikitable(s) on the page` (earlier run)
Turn the collected rows into a DataFrame	`rows`	`scraped_df`: the inverse codon table (Amino acid, RNA codons, Compressed, ...)

The row loop is given; only the DataFrame step is a TODO. A final cell strips reference markers like `[1]` with a regular expression and saves `scraped_table.csv`. It uses `DataFrame.map` (the old `applymap`

was removed in pandas 3) with a fallback for older pandas.

What can go wrong. HTTP 403: the `User-Agent` header was removed. If Wikipedia changes the page, the number of tables or the index `tables[2]` can change: print `len(tables)` and pick the table again.

★ Part 5: Headless Selenium (optional, about 10 min)

Optional because installing Chrome inside Colab takes 2–4 minutes. The instructor demos a visible browser; the notebook runs the same steps headless: open Wikipedia's search page, type "Ribonucleic acid" and Enter, wait for the article, print its first long paragraph. Earlier run: `DEMO SUCCESSFUL! Landed on: RNA`. No TODO.

What can go wrong. The install uses `apt-key`, which newer Colab images may reject. If it fails, participants simply watch the demo; nothing later depends on this part.

Part 6: Capstone, a reusable pipeline (about 10 min)

Aim. Turn Part 2.3 into a function: search term in, cleaned DataFrame out. The function requests the data, builds the rows, then removes duplicates, drops rows without an ID or length and fills a missing `rna_type` with `unknown`.

TODO	Participants type	Expected result
Fill the two row fields inside `build_rna_dataset`	`r.get("length")` and `r.get("rna_type")`	`Function build_rna_dataset is ready to use.` then, for "miRNA precursor", `Retrieved 30 clean records` (earlier run); saves `session1_rna_dataset.csv`

Point out that running the definition cell prints nothing useful: defining a function is not the same as using it.

> **Instructor note.** In the earlier run this search returned the same records as Part 2.3, which confirms the RNAcentral query is ignored. A "try your own search term" exercise was deliberately not added for this reason; add it once the query is fixed.

★ Part 7: Text and tables from a PDF with Docling (optional, about 10 min)

Optional because Docling downloads several models on first use (3–5 minutes). The notebook downloads "Attention Is All You Need" (arXiv 1706.03762), converts it to Markdown and lists its tables. One TODO: pass `pdf_path` to `converter.convert`. Earlier run: the first 2,000 characters of Markdown, then `Found 4 table(s) in the document` and the first table as a DataFrame. If Docling fails, the cell falls back to plain text with pypdf (no tables).

Common problems in Session 1

Symptom	Cause	Fix
`NameError: name '___' is not defined`	A blank was left in a TODO	Fill the blank (or reveal the answer) and re-run the cell
`NameError` for another variable	A cell above was not run	Runtime → Run before
`HTTPError 429` or `5xx`	Rate limit or a busy server	Wait 10–30 s and re-run
RNAcentral returns unrelated RNAs	The `query` parameter is ignored	See the note in Part 2.3
Wikipedia table looks wrong	Page layout changed	Print `len(tables)`; choose another index
Selenium or Docling install fails	Colab image changes, slow downloads	Skip: both parts are optional

Session 2: Machine learning on RNA data

Detailed walkthrough · `Session2_Machine_Learning_RNA.ipynb`

Big picture

Aim. Build a coding vs. non-coding RNA classifier from start to finish: data → features → training → validation → evaluation → interpretation. The statistics section (t-test and Cohen's d) was removed so the session concentrates on basic machine-learning tasks. About 60 minutes core, 10 minutes optional. No GPU.

What participants do. Thirteen core TODOs and three optional ones, each a single blank: a column name, a number such as `0.2` or `5`, a function or variable name. Every scikit-learn step uses the same *create* → *fit* → *predict* pattern, which the notebook says out loud.

What they should leave with. (1) Features are numbers we choose to describe a sequence. (2) A model is judged on data it has never seen; a big train/test gap means overfitting. (3) One accuracy number is not enough: look at the spread across folds, precision and recall, the confusion matrix and the ROC curve. (4) Feature importance says what the model relied on, which can be checked against biology.

Part	Minutes	What happens
Setup	3	Install and import pandas, NumPy, matplotlib, scikit-learn
1 Problem types	7	Three toy datasets: classification, regression or clustering? (★ K-Means)
2 Prepare data	15	Download, check, clean, engineer 3 features, encode, scale, split 80/20
3 Train	10	Decision tree (overfitting), logistic regression, random forest
4 Cross-validation	7	5-fold CV (★ GridSearchCV tuning)
5 Evaluate	12	Accuracy, precision, recall, F1, confusion matrices, ROC/AUC
6 Interpret	5	Random-forest feature importance
Save	2	Two CSVs for the homework
★ Bonus	10	Optional: K-Means on the real data, Adjusted Rand Index

A short glossary cell at the top defines *feature*, *label*, *training set / test set* and *overfitting*.

Part 1: Classification, regression or clustering? (about 7 min)

Three plots, no code to write. Dataset A: sequence lengths labelled coding or non-coding. Dataset B: GC content against a continuous expression level. Dataset C: two unlabelled groups. A collapsed cell reveals the answers: **A classification, B regression, C clustering**.

★ **Optional TODO:** K-Means on dataset C, `n_clusters=2` (participants type `2`). The plot shows two coloured groups with red centres.

The two toy-plot cells for A and B were scrambled in the original notebook and would not run; they are rebuilt here.

Part 2: Clean and prepare the RNA dataset (about 15 min)

2.1 Download (no TODO)

`fetch_noncoding` asks RNAcentral for 100 sequences; `fetch_coding` asks NCBI for 100 human RefSeq mRNAs of 200–3,000 nt and parses the FASTA. If anything fails or fewer than 60 records arrive, `synthetic_fallback` builds 200 random sequences with deliberately overlapping lengths and slightly different base composition. Output: `Fetched 200 real sequences from RNAcentral + NCBI` or `Live fetch failed ... using synthetic fallback dataset instead`. Either way the session continues.

> **Instructor note.** The markdown explains the task as mRNA vs. lncRNA, but with the current RNAcentral request the non-coding class is whatever RNAcentral returns by default (short tRNA, rRNA and misc_RNA records in the earlier run). That makes length an easy separator and explains the high accuracies below. Fix the request or adjust the text before sending.

2.2–2.7 Check, clean, features, encode, scale, split

Step	TODO: participants type	Expected result
2.2 Count each label	`"label"`	`(200, 2)`; 100 coding, 100 non-coding; 0 missing values
2.3 Remove duplicate sequences	`"sequence"`	`200 -> 200 rows after cleaning`; sequences converted to RNA alphabet
2.4 Longest ORF for every sequence	`longest_orf_length`	New columns `length`, `gc_content`, `orf_length`, then three overlapping histograms
2.5 Encode the label	`clean_df["label"]`	`{'coding': 0, 'non-coding': 1}`
2.6 Scale features	no TODO	Three `_scaled` columns, mean 0 and standard deviation 1
2.7 Train/test split	`0.2`	`Train: 160 rows | Test: 40 rows`, class balance `{0: 80, 1: 80}`

Debrief. Why describe a sequence with numbers? Why should a long open reading frame point to a coding RNA? Why is the test set hidden, and why `stratify`?

Part 3: Train the classifier (about 10 min)

The notebook states the scikit-learn recipe: create the model, `fit(X_train, y_train)`, `predict(X_test)`.

TODO	Participants type	Reference result (earlier live run)
Fit a single decision tree	`X_train` and `y_train`	Train accuracy 1.000, test accuracy 0.900
Number of trees in the random forest	`200`	Logistic regression test 0.925; random forest train 1.000, test 0.900

Debrief. A perfect training score is a warning sign. In the earlier run the forest's gap was no smaller than the tree's: say so honestly. With 40 test sequences, one sequence is 2.5 percentage points, so small differences mean little. That leads straight into cross-validation.

If the synthetic fallback was used (offline test run): tree test 0.775, logistic regression 0.875, forest 0.900; here the tree overfits visibly.

Part 4: Cross-validation (about 7 min)

TODO	Participants type	Reference result (earlier live run)
Number of folds	`5`	Scores [0.975, 1.0, 0.975, 0.875, 1.0]; mean 0.965, std 0.046 (fallback data: mean 0.860)

★ **Optional: GridSearchCV.** No TODO, just run: 4 depths × 3 forest sizes × 5 folds = 60 fits, about 30–60 s. Earlier run: best `{'max_depth': 5, 'n_estimators': 100}`, CV 0.981, test 0.950. If this cell is skipped, Part 5 automatically uses the default forest and labels it "tuning skipped".

Debrief. How far apart are the five fold scores? Did tuning help, or was the default already close?

Part 5: Evaluate the models (about 12 min)

A first cell makes sure a `best_rf` exists. The text defines accuracy, precision, recall and F1, and warns that `LabelEncoder` numbered the classes alphabetically: coding = 0, **non-coding = 1, so precision and recall describe how well the non-coding class is found**.

TODO	Participants type	Reference result (earlier live run)
F1 inside `evaluate()`	`f1_score`	Logistic regression 0.925 / 0.905 / 0.950 / 0.927; forest (default) 0.900 / 0.944 / 0.850 / 0.895; forest (tuned) 0.950 on all four (accuracy / precision / recall / F1)
AUC on the ROC plot	`y_proba`	Two ROC curves above the dashed diagonal, AUC in the legend

Confusion matrices (one per model) are drawn without a TODO: the diagonal is correct predictions.

Debrief. Which model would you trust, and why? What does a false negative (a coding RNA missed) cost compared with a false positive? Does the ROC curve change the answer?

Part 6: Interpret what the model learned (about 5 min)

TODO	Participants type	Reference result (earlier live run)
Label the importances	`feature_cols`	orf_length 0.43, length 0.42, gc_content 0.14

Debrief. Look back at the Part 2.4 histograms: does the feature with the most class overlap get the lowest importance? Does the ranking fit coding-sequence biology (long ORFs)?

Save results

Writes `session2_rna_dataset_clean.csv` and `session2_model_results.csv` and downloads both in Colab (the browser may ask permission for multiple downloads). Outside Colab it just saves them.

★ Bonus: unsupervised clustering of the real data (optional)

TODOs: `2` and `X` for K-Means, then `cluster_labels_real` for the Adjusted Rand Index. A cross-table compares clusters with the true labels. In the offline fallback run, ARI was 0.11: K-Means mostly did *not* recover the classes. Debrief: features that help a supervised model can still be poor for clustering, which never sees the labels.

Common problems in Session 2

Symptom	Cause	Fix
"Live fetch failed ... synthetic fallback"	RNAcentral or NCBI unreachable or slow	Fine to continue; numbers differ from the reference
`NameError: best_rf`	Part 5's first cell skipped	Run the cell that starts "If the optional GridSearchCV step was skipped"
`ValueError` in `train_test_split`	`test_size` not a number between 0 and 1	Type `0.2`, without quotes
Downloads blocked	Browser blocks multiple downloads	Allow pop-ups/downloads for colab.research.google.com
Numbers differ from a neighbour	Live data changed between runs	Normal; compare the pattern, not the decimals

Don't take the number at face value

Two structure predictors, one impossible protein · detailed walkthrough of `Model_Confidence_Workshop_ESMFold_AlphaFold.ipynb`

Big picture

Aim. Teach participants to read the confidence scores of protein structure predictors, and to test them. A real protein domain is turned step by step into a sequence that cannot fold, and the confidence reported by two independent models is compared with what chemistry says. About 70 minutes. Designed for a mixed wet-lab and computational audience; no coding background needed.

The claim. Not "AI models are bad" and not "AlphaFold hallucinates". The narrower claim: *a model can hand you a confident-looking number that does not answer the question you are asking, and the way to find out is to check.*

The models. AlphaFold2 (with a full multiple sequence alignment), run beforehand with ColabFold; its numbers are pasted in. **ESMFold** (`facebook/esmfold_v1` via Hugging Face transformers), run live on a T4 GPU from a single sequence, using a protein language model.

The two scores. pLDDT (0–100, per residue): predicted local accuracy; above 70 is conventionally confident. **pTM** (0–1, whole protein): is the overall fold right? Below 0.5 is conventionally "probably wrong".

What participants do. Five core TODOs (choose a mutant, look up a score, set two thresholds, fold their own variant, pick a structure to view) and two optional ones, plus a written prediction before ESMFold runs.

What they should leave with. (1) pLDDT and pTM answer different questions. (2) A confidence score is about the task the model was trained for (accuracy on proteins that fold), not stability. (3) A second, independent model is a check. (4) Build a case where you already know the answer, and see whether the number tracks it.

Part	Minutes	What happens
0 Setup	5	GPU check, install, load ESMFold (runs while participants read)
1 Sequences	10	GB1 wild type, 0–10 core aspartates, shuffled and reversed controls; your own variant
2 AlphaFold2	5	Pre-computed pLDDT and pTM for the 8 sequences
3 Two numbers	10	Apply the thresholds; pLDDT and pTM disagree on core_10D
4 ESMFold	15	Predict first, fold all 8 live (or backup), fold your own variant
5 Agree?	10	Side-by-side table and bar chart, 3D structures coloured by pLDDT
6 Discussion	15	What we can and cannot conclude; questions for the room
★ Extras	–	Per-residue pLDDT, alanine series, save results, why fp16

Part 0: Setup (about 5 min)

Before running. Runtime → Change runtime type → T4 GPU. Participants run the cell first and read Parts 1–3 while it loads.

What the cell does. Installs `transformers accelerate py3Dmol`, picks the GPU, loads the tokenizer and the model in half precision (fp16) to keep RAM low, then switches everything except the language model back to fp32 (the folding trunk and the pTM head give NaN in fp16), moves the model to the GPU and sets `trunk.set_chunk_size(64)`. Success prints `✔ ESMFold loaded in Ns` (53 s in the earlier run) and sets `MODEL_OK = True`.

If it fails. No GPU, no quota or out of memory: the cell prints a friendly message, `MODEL_OK` stays False, and Part 4 uses saved ESMFold results. Warnings about `HF_TOKEN` or MISSING `esm.contact_head` parameters are harmless.

Part 1: The sequences (about 10 min)

GB1, the 56-residue B1 domain of protein G: a four-stranded β-sheet packed against one helix, with a textbook hydrophobic core (Y3, L5, L7, F30, A34, V39, W43, Y45, F52, V54). These positions are mutated to **aspartate** two at a time; burying a negative charge away from water is very costly, and by `core_10D` the sequence cannot fold. Two controls keep the same composition: **shuffled** (seed 0, same as the AlphaFold2 run) and **reversed**. Every core mutant stays at least 82% identical to wild type.

Variant	core_00D	core_02D	core_04D	core_06D	core_08D	core_10D	shuffled	reversed
% identity	100.0	96.4	92.9	89.3	85.7	82.1	23.2	10.7

TODO	Participants type	Expected result
How many core residues to mutate	`5` (any 0–10)	`my_variant` printed; 91.1% identity for 5

Debrief. Why aspartate? Why is high sequence identity to wild type the point of the experiment?

Part 2: AlphaFold2 results (about 5 min)

Explain why these numbers are pre-computed (a live MSA search per sequence would take too long) and that AlphaFold2 had every advantage: a full multiple sequence alignment.

	core_00D	core_02D	core_04D	core_06D	core_08D	core_10D	shuffled	reversed
AF2 pLDDT	94.8	94.0	86.5	88.0	79.9	76.9	45.5	39.7
AF2 pTM	0.767	0.749	0.619	0.640	0.521	0.489	0.207	0.250

TODO	Participants type	Expected result
Look up one variant	`"core_10D"`	`core_10D: pLDDT = 76.9 pTM = 0.489`

Part 3: One model, two numbers, two answers (about 10 min)

TODO	Participants type	Expected result
Set the two conventional thresholds	`70` and `0.5`	A verdict for each variant; only `core_10D` is flagged "the two metrics DISAGREE" (pLDDT confident, pTM fold probably wrong)

The plot shows pLDDT and pTM×100 against the number of buried aspartates, with both thresholds and the shuffled "no idea" floor at 45.5.

Key message. Same model, same prediction, same sequence: the headline number (pLDDT 76.9) says fine, the global-fold number (pTM 0.489) says probably wrong. A number labelled "confidence" has a specific meaning, and reading it as a general endorsement is the user's error. core_10D sits about 31 points above the shuffled floor.

Part 4: ESMFold (about 15 min)

First, why a second model is informative: ESMFold uses no alignment, only the single sequence and a protein language model, so it is an independent read, not a second opinion from the same evidence. Participants then write a prediction in a text cell (higher / similar / lower than 76.9 for core_10D, and why).

Folding cell. Defines `esmfold(seq, name)` (mean pLDDT, pTM, per-residue pLDDT and a PDB string; a NaN check catches a half-precision problem) and folds the 8 sequences in about 10–20 s.

	core_00D	core_02D	core_04D	core_06D	core_08D	core_10D	shuffled	reversed
ESM pLDDT	89.0	64.2	52.4	50.6	48.2	55.0	49.2	43.5
ESM pTM	0.761	0.454	0.316	0.278	0.111	0.110	0.195	0.194

Backup cell. Safe to run either way: if any of the 8 results is missing, it fills them from the table above (from the earlier run). Backup results have no 3D structure or per-residue scores.

TODO	Participants type	Expected result
Fold your own variant	`my_variant`	One line with its pLDDT and pTM; compare with its neighbours in the series. Skipped with a message if the model is not loaded.

Part 5: Do the two models agree? (about 10 min)

A table with AF2 and ESM pLDDT, their difference and both pTMs, and a grouped bar chart against the 70 line. In the earlier run ESMFold's confidence fell steeply already at core_02D (64.2, a drop of 29.8 against AF2) and stayed near the shuffled level from core_04D on; its pTM for core_10D (0.110) was below the shuffled control's.

TODO	Participants type	Expected result
Choose a second structure to view next to wild type	`"core_10D"` (or "shuffled", "my_variant")	Two rotatable 3D cartoons coloured by pLDDT (red low, blue high)

Debrief. Where did the models agree and disagree? Which is closer to the chemistry for core_10D? What would you have concluded with only one model? One explanation to offer for discussion, not as a finding: AlphaFold2's alignment of natural GB1 relatives may keep its confidence up even when the core is destroyed, while the single-sequence model has nothing to lean on.

Part 6: What this does and does not show (about 15 min)

We are entitled to say	We are not entitled to say
A sequence that cannot fold did not get a "cannot fold" answer from AlphaFold2 (76.9, 31 points above its shuffled floor).	"AlphaFold2 is broken." It was trained on the PDB, only proteins that fold; pLDDT is predicted accuracy, not stability.
The two confidence metrics disagreed; the headline one said fine.	"The model is blind to chemistry." The AF2 curve does fall, 94.8 to 76.9; it understated the effect.
Whatever the second model did is data: agreement strengthens the finding, disagreement shows two AI answers that cannot both be right.	"This is about LLMs." AlphaFold2 is not a language model; ESMFold contains one.

The lesson. A confidence score reports expected performance on the task the model was trained for. Read as an answer to another question it only looks like evidence. Catch this by building a case where you know the answer and checking whether the number tracks it.

Questions for the room. What would measure foldability (Tsuboyama et al. 2023, mega-scale stability data)? Is shuffled's 45.5 chemistry or an empty alignment? If two models disagree, what breaks the tie? Where in your own work do you read a confidence score as an answer to a question the model was not trained on?

Terminology. In protein design, "hallucination" means deliberately designing a sequence to maximise a network's confidence (Anishchenko et al. 2021). Call what we saw *miscalibrated confidence* or *out-of-distribution failure*.

★ Optional extras

- **Per-residue pLDDT** (TODO: `"core_10D"`): two lines along the chain with the 10 core positions shaded red. Needs a live ESMFold run.
- **Alanine series** (TODO: `"A"`): folds core_00A, core_04A and core_10A. Alanine removes the packing but buries no charge; does confidence fall less steeply than for aspartate? Needs ESMFold.
- **Save results**: writes `both_models.pkl` with both models' scores and the ESMFold structures.
- **Under the hood**: why the model is loaded in fp16 and partly converted back to fp32.

Limitations stated in the notebook. AlphaFold2 without an alignment is not a usable control (wild type 58.5, core_10D 61.2 in an earlier test). The AF2 numbers used 1 model and 1 recycle, hence the 4→6 wobble. n = 1 protein: a demonstration, not a study.

Common problems in the model-confidence notebook

Symptom	Cause	Fix
"No GPU" message in Part 0	CPU runtime or no GPU quota	Change runtime to T4 GPU and re-run; otherwise continue with the backup
Session crashed, all RAM used	Model loaded in fp32 or another notebook open	Restart the session and run Part 0 again
`NaN in pLDDT`	Part of the model still in fp16	Re-run the setup cell
3D viewer says "No 3D structure available"	Backup results in use	Expected without a live ESMFold run
`KeyError: 'my_variant'`	The fold-your-own cell was skipped	Choose another variant in the viewer

Instructor checklist

Before sending the notebooks and before each session

A week before

1. Decide on the RNAcentral search (fix the request, or teach it as a "check what came back" moment) and update Session 2's mRNA vs. lncRNA text to match.
2. Run each answer key top to bottom in a fresh Colab runtime. Write your own numbers in the table below.
3. Test Session 1's RCSB POST cell.
4. Confirm a T4 GPU is available on the Colab accounts participants will use.
5. Send participants only the three participant notebooks, with a request to open each in Colab, save a copy to Drive and run the Setup cell the day before.

Values to verify

Notebook	Check	Reference	Your run
Session 1	MIR21 gene ID / APP length	406991 / 770	
Session 1	Wikitables found	8	
Session 1	RCSB POST status	200 (untested)	
Session 1	Capstone records	30	
Session 2	Rows and split	200; 160 / 40	
Session 2	Random forest test accuracy / 5-fold CV mean	0.900 / 0.965	
Session 2	Top feature importance	orf_length 0.43	
Model confidence	ESMFold load time	53 s	
Model confidence	ESM pLDDT core_00D / core_10D	89.0 / 55.0	

On the day

- Start each session by having everyone run Setup while you introduce the topic.
- Remind participants that revealing an answer is fine; a blank left in a TODO breaks later cells.
- For the model-confidence session, read the "what we are not entitled to say" list out loud, and collect a few written predictions before running ESMFold.
- Numbers that differ slightly from a neighbour's are normal (live data, CPU vs. GPU).